

Programming project report: DPLL algorithm in Why3

Savan Kan

January 30, 2026

1 Inefficiency

For the dpll algorithm implementation i decided to put in priority ease of specification and ease of verification over efficiency. Indeed, the implementation for the dpll that i considered is not very efficient and you can actually witness that by running the provided tests. The inefficiency is mostly due to my implementation of the `scan` function which consider a clause $C = \ell_1 \vee \ell_2 \vee \ell_3$ to be satisfied with respect to an assignement M when not only one of the ℓ_i is satisfied by M but also all those literals has to be defined for M . That is in my implementation C is considered satisfied when all the ℓ_i are defined for M and one of them is satisfied for M .

For instance the clause $x \vee y \vee z$ will not be considered satisfied (even though it should of course) by the assignment $[x \mapsto 1, y \mapsto 1]$ since it is undefined for z . I could have of have redone the whole implementation in a more efficient way but i couldn't due to lack of time.

2 Definedness predicates

In my implementation, not only we have computational version of $(\text{ok_cls } M \text{ na } C)$ and $(\text{ok_lit } M \text{ na } \ell)$ (which are respectively `okc` M C na and `okl` M na ℓ); there is two predicate that we use: `ldef_pr` M ℓ that tells us that the literal ℓ is defined in the sense that $M[[\ell]]$ is either equal to ℓ or $-\ell$. And `cdef_pr` M C that tells us that a clause is defined in the sense that all its literals are defined in the sense mentioned. Those predicates have computational versions: the functions `ldef` and `cdef`.

There is also the functions `strong_ldef` and `strong_cdef` that are here for talking about “strong definedness”. Precisely, i call a literal strongly defined when it is both defined in the previously mentionned sense but also $|\ell| < \text{na}$ holds. And i call a clause strongly defined when all its literals are strongly defined.

3 The scan function

Here is the `scan` function without annotations:

```
1 let scan (mm: assignment) (cl: array cls) (nv na nc: int)
2 : (b: bool, mc: int)=
3
4 let ref i = 0 in
5 let ref b = true in
6 let ref mc = nc in
7 for i=0 to length cl -1 do
8   while i<mc do
9     if strong_cdef mm na cl[i] then
10       if okc mm cl[i] na then (mc <- mc-1; swap cl mc i)
11       else (b <- false; break)
12     else break
13   done
14 done ;(b, mc)
```

Contract: The function requires:

- All the elements of the array cl to be indeed clauses through the predicate `is_cls` $|M|$ $cl[i]$.
- Bounds: $nv \geq 0$, $0 \leq na \leq |M|$, $0 \leq nc \leq |cl|$
- All clauses of index in the range $[nc..|cl|)$ are defined and satisfied.

And it ensures the invariants that i will list later.

How it works: Here is a not too detailed explanation of how it works, we start with index $i = 0$ and check if the clause $cl[i]$ is undefined (in the sense that one of its literal is undefined for the assignment mm) or unsatisfied (in the sense that all its literals are defined and all of them is unsatisfied by mm); if one of those two things holds then we move on to the next iteration of i ; otherwise, the clause $cl[i]$ is both defined and satisfied so we do the swapping and that swapping might result in a situation where the new $cl[i]$ is also both defined and satisfied in which case we do again a swapping until we either finally get out of that situation or we reach a state where $i \geq mc$; and then we proceed to the next iteration of i etc.

Loop dependency: If you take a look at the invariant annotations for this function in the `dp11.mlw` file, you will notice that for each invariants of the algorithm that are considered, we specify these invariants as both invariants for the for-loop and the inner while-loop at the same time. The reason is to get that a condition I is an invariant for our outer loop (the for loop), we need it to be preserved after traversing the while loop and in order to get that preservation we need to require I to be an invariant of the while loop as well.

And also, note that in order for an invariant I of the while loop to initially hold, we need to make I holds for every index i i.e. we need I to be an invariant of the for-loop. So there is this interesting dependency which requires us to have to specify an invariant I to be an invariant of both loops even if we want it to be specified as invariant for only just one of the loops. Maybe there is a way to avoid this duplication of annotation which i don't know. There can be however some simple enough invariant which do not need this dependency. In the code there is specifically one such invariant:

```
1 invariant {(b=true/\ mc=0) -> forall k. 0<=k < length cl ->
2         cdef_pr mm cl[k] /\ ok_cls mm na cl[k]}
```

This for-loop invariant does not need to be specified as invariant for the while loop because it can be proven to be an invariant as a consequence of another for-loop invariant, namely the following:

```
1 invariant {forall j. mc<=j <= length cl -1 -> cdef_pr mm cl[j]/\
2         ok_cls mm na cl[j]}
```

Considered invariants:

1. If the returned boolean b is false then there is at least one unsatisfied clause: obviously holds by construction.
2. The current and old clause array cl are permutation of each other: We did not introduce any new clauses, we just swapped clauses so in the end cl is a permutation of the old one. This is needed for the `dp11` function which in turn is needed to prove one of the requirement of the `sat` function.
3. The current and old clause array cl agree for indexes $i \geq nc$: this is indeed the case since we always do the swapping within the dynamically decreasing active zone $[0..mc)$ and never do anything at what's above. That invariant is required for ensuring the precondition of satisfaction of the inactive clauses for some calls of `scan` within the `dp11` function.

4. All clauses of index $i \geq mc$, are all defined and satisfied by the assignement: if $i \geq nc$ this is obviously true since its a precondition of `scan`, otherwise if $mc \leq i < nc$ then $cl[i]$ is satisfied and defined because it's supposed to be obtained through a swap with such a clause.
5. We have $0 \leq mc \leq nc$: mc is initially nc and always decreases and we can be sure that it never gets below 0 since we decrease it within a while loop guarded by the condition $(0 \leq i < mc)$.
6. If `scan` returns $(true, 0)$ then all clauses in cl are both defined and satisfied by mm : like mentioned before, any clauses of index i such that $mc \leq i < nc$ has to come from a swapping with a defined and satisfied clauses, so all clauses in this range are satisfied and since $mc = 0$ here, we have that every clause in the range $[0..nc)$ is both defined and satisfied, moreover the precondition of `scan` entails that it is the same for clauses in the range $[nc..|cl|)$ so all the clauses are defined and satisfied. We note that this invariant also holds when `scan` returns $(false, 0)$ as we did not use any information about the returned boolean so we need to be cautious when using `scan`.

4 The `dpll` function

Here is the `dpll` function without annotations:

```

1 let rec dpll (mm: assignment) (cl: array cls) (nv na nc: int)
2 : (s: bool)=
3
4 let mm1 = mycopy mm in
5 let mm2 = mycopy mm in
6 if na < length mm then
7   begin
8
9     mm1[na] <- na;
10    let b,mc = scan mm1 cl nv (na+1) nc in
11    if b && mc=0 then (build_asgn mm1 mm (na+1); true) else
12
13      if dpll mm1 cl nv (na+1) mc then ( copy_into mm1 mm ; true)
14      else (
15        mm2[na] <- -na;
16        let b',mc' = scan mm2 cl nv (na+1) nc in
17        if b'&& mc'=0 then (build_asgn mm2 mm (na+1); true) else
18        if dpll mm2 cl nv (na+1) mc' then (copy_into mm2 mm; true) else false
19      )
20
21    end
22 else
23   (
24     let b,mc = scan mm cl nv na nc in
25     if b && mc=0 then true else false
26   )

```

The function basically works as described in the project description/énoncé.

Note that since the code for `dpll` is a bit more involved in terms of technicalities compared to `scan`, to prove the validity of `dpll`, you should run `Auto level 2` to prove it.

Contract: The function requires:

1. mm is an assignment defined up to na .
2. Bounds: $|mm| > 0$; $nv, na \geq 0$; $0 \leq nc \leq |cl|$
3. All the elements of the array cl to have to be indeed clauses.
4. All clauses of index in the range $[nc..|cl|)$ are defined and satisfied.

Most of these requirement are here so that we can use **scan**. And the first requirement is mostly here so we can ensure in the end that we obtain an assignment defined up to $|mm|$ which also required me to add an extra precondition “ $mm[0] = 0$ ” for the **sat** function in order to make sure that this assignment precondition of **dp11** is satisfied in the **sat** function.

What the **dp11** function ensures are the desired postconditions for the **sat** function except the non-existence of models (because i don’t have enough time to work on it).

The **dp11** function also ensures some preservation properties so that we can prove certain preconditions for **scan** and the other used utility functions after a recursive call, namely it ensures that: the length $|mm|$ never changes, the elements of cl are still indeed clauses, and clauses of index in range of $[nc..|cl|)$ stays the same just like for the **scan** function.

Used utilities:

- **copy_into** $M_1 M_2$: it takes two assignments of same length and copy the elements of M_1 into M_2 componentwise. Note that this function has the preservation property: if M_1 is an assignment up to na then so is M_2
- **fill_from** $M na$: it takes an assignement M and fills all slots of index $i \geq na$ with “true”. It ensures two preservation properties: if the old M is an assignement defined up to na then so is the current M , and if the old M satisfies a clause C then the current M also satisfies this C . It also ensures that in the end, the assignment is defined for every indexes.
- **build_asgn** $M_1 M na$: it uses the two utilities above to build a complete extension of M_1 as M while preserving satisfaction of clauses.

These utilities are here for basically ensuring that in the end if **dp11** returns *true* then the current assignment M is defined for all of its indexes: we may have an assignment not entirely defined that satisfy all the clauses of the 3CNF formula, let’s say:

$$M = [something, something, undef, undef, undef]$$

so to ensure that in the end we have the post condition that the current assignment has to be wholly defined when **dp11** returns *true*, we do some padding by adding a dummy truth value (here “true”) to complete it:

$$M = [something, something, dummy, dummy, dummy]$$

And this does not break the satisfaction of the clauses nor the 3CNF formula due to their structures. Note that we use this padding to M in the **dp11** only when we know that M satisfy the 3CNF formula.

5 Conclusion

In this report, the implementation, specifications and verification of the core functions `scan` and `dp11` were discussed.

In the end, the resulting outcome was a bit unsatisfactory due to the inefficiency of the implementation and the fact that the non-existence of model was left unproven due to lack of time. But for the non-existence of model one possible path for trying to get a proof for it would be to add a precondition to `dp11` stating that “all valid assignment mm' defined up to na must agree with the current assignment mm on the index range $[0..na)$ ” and try to see what tweaks we need to introduce in order to get the desired proof for non-existence of models.

I sometime invoked Rocq within Why3 for the proofs but it actually wasn't needed as everything was doable with just the alt-ergo prover. But invoking it allowed me to understand a bit of how some things work in Why3.

I think the main difficulty was to determine how much we should constraint the various preconditions so that we can prove the invariants and other properties without ending up with something too constrained which would make things very hard to prove. I think finding the right balance was not an easy task and keeping track of all the information was also a bit painful.

Working on this project allowed me to get familiarity with deductive provers and SMT solvers as well as Hoare logic so i really enjoyed working on the project.